

# ***"I'm Looking for an Airfoil, Mr. Anderson": Machine Learning Surrogates and Reinforcement Learning for Aircraft Design***

**Aerospace innovations driven by Artificial Intelligence  
Politecnico di Torino 2026**

**Arnau Miró ([arnau.miro@upc.edu](mailto:arnau.miro@upc.edu))**

# About today's lecture

## Research questions

- Can a lightweight Machine Learning model trained on simple geometric parameters accurately predict the lift and drag coefficients of an unseen airfoil in milliseconds?
- At what point does an AI surrogate model begin to fail when predicting flow behaviour outside of its training envelope?
- How does the optimization workflow of a Reinforcement Learning agent change when it is tasked with modifying a static physical blueprint (Aerodynamic Shape Optimization) versus responding dynamically to a live fluid environment (Active Flow Control)?
- What are the trade-offs in computational training time and predictive accuracy when using Reinforcement Learning as an optimization framework compared to traditional, physics-only engineering methods?

# About today's lecture

## Roadmap

Act I: What an answer costs

Act II: Learning the model

Act III: How much can I trust the model?

Act IV: Using the model

Act V: RL, what it finds, where it wins

Act VI: Final thoughts

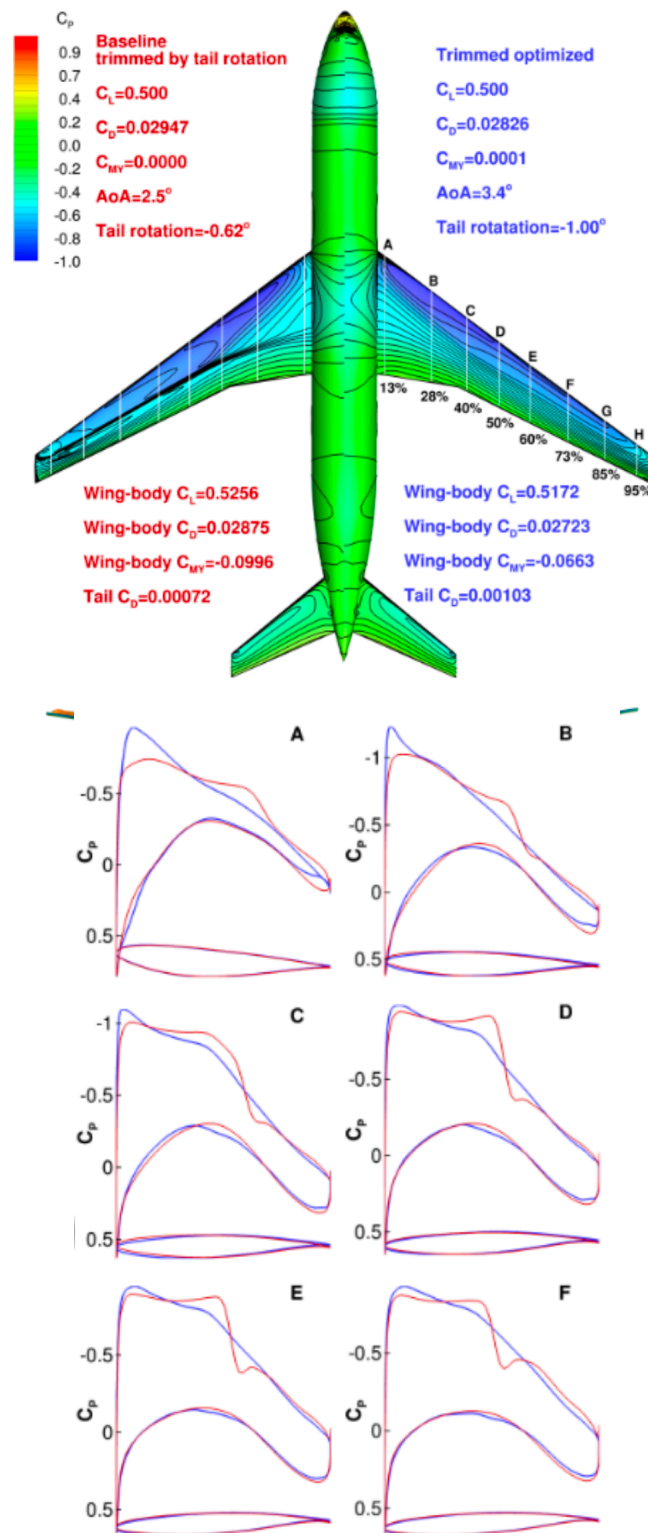
*Act VII: practical sessions tomorrow afternoon linking all!*

**Act I: What an answer  
costs**



# Act I: What an answer costs

## Conceptual design: cost *committed* vs cost *incurred*

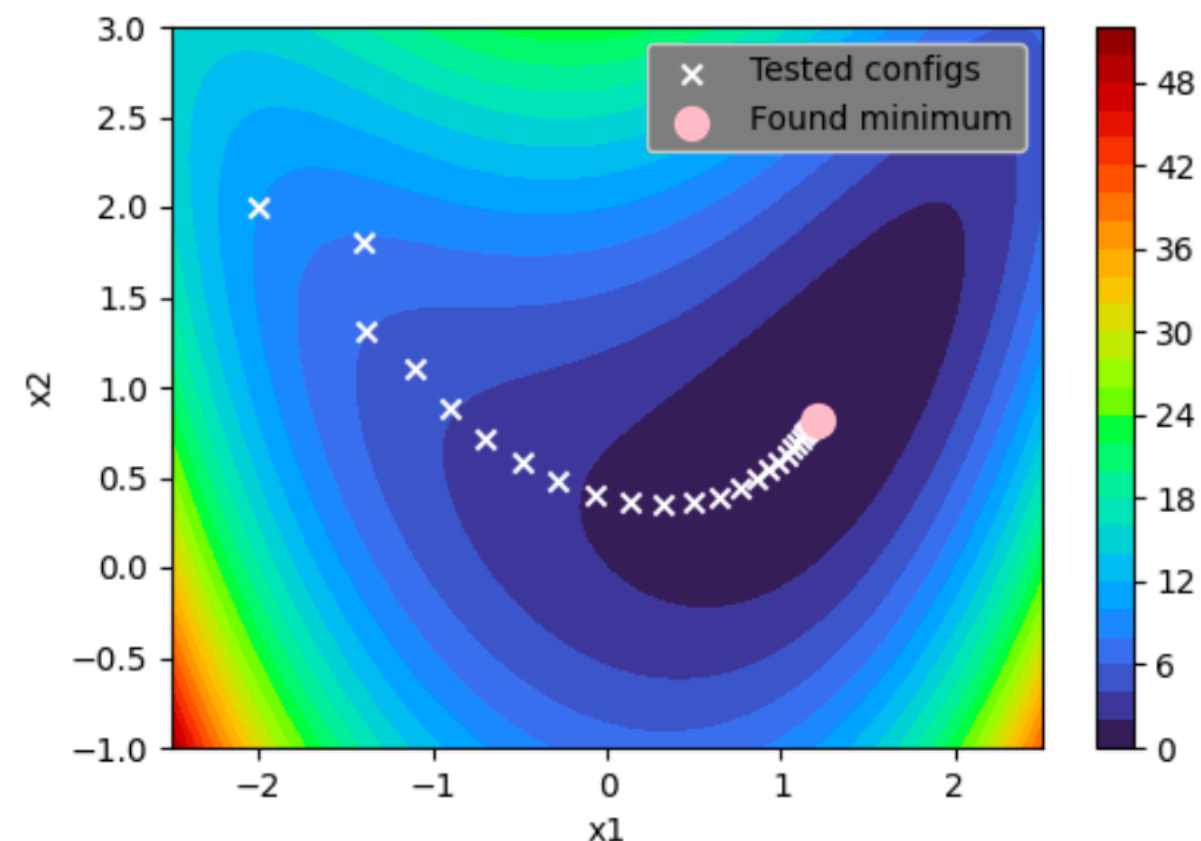


- Were all the presented optimizations using the **same methodology** to pick the best candidate? and to **model** the flow?
- Is the **number of design variables** important? Do we always have a **single objective function** to optimize?
- Were these optimizations something I can do with my laptop in an afternoon? Are there any ideas to reduce the overall **computational cost** of the optimization?
- How were **shape modifications** taken into account?
- Was the **aerodynamic performance the only metric** they look at?
- How do we **check the provided result is physical**?

# Act I: What an answer costs

## The design/optimization loop

- Given a set of points  $x_i$ , evaluate the objective function  $f$  to obtain  $y_i$
- Obtain a measure of how “good” the evaluation  $y_i$  is from the minimum
- Generate a new set of points  $x_i$  to evaluate  $f$ , rinse and repeat
- The amount of parameters  $x_i$  are called *design variables* in an optimization context



# Act I: What an answer costs

## Evaluations per search strategy

### Gradient-based methods

- $\nabla f$  must exist, is known and can be evaluated (reasonably)
- Fast approach towards the minimum with minimal evaluations: 10 to  $10^2$
- Methods are generally *mathematical*

### Gradient-free methods

- $\nabla f$  is unknown and only  $f$  can be evaluated
- We can afford a reasonable number of evaluations of  $f$ : up to  $10^4$
- Methods are generally *heuristic*

# Act I: What an answer costs

## The evaluation of $f$ : aerodynamic solvers

Given an aerodynamic optimization problem, we need to find a suitable model bearing in mind (**recall the number of evaluations of the objective function**) the level of *fidelity/efficiency*:

- *Low fidelity and high efficiency*, related to simpler models:
  - Inviscid/Potential flow with corrections for compressibility and viscous effects
  - Tuned coarse RANS with wall models models for massively separated flows (with their limitations)
- *High fidelity and low efficiency*, related to complex models:
  - Viscous RANS/LES without wall models (even DNS!) in simpler geometries and moderate Re.
  - Viscous wall model LES in complex geometries or high Re.

# Act I: What an answer costs

## The evaluation of $f$ : cost

| Parameter              | XFOIL                                | CFL3D (RANS)                | SOD2D (LES)                  |
|------------------------|--------------------------------------|-----------------------------|------------------------------|
| Method                 | Panel + viscous/inviscid interaction | Thin-layer URANS            | WMLES                        |
| Spatial discretization | Panel method                         | Finite Volume               | Spectral Continuous Galerkin |
| Grid                   | 301 panels                           | Structured                  | Spectral-element             |
| Reynolds number        | $3 \times 10^6$                      | $3 \times 10^6$             | $3 \times 10^6$              |
| Mach number            | 0.15                                 | 0.15                        | 0.15                         |
| Transition model       | $N_{crit} = 7$                       | Fully turbulent             | WMLES                        |
| Maximum iterations     | 201                                  | Implicit convergence        | Statistical convergence      |
| Turbulence model       | Integral boundary layer              | Spalart–Allmaras            | Vreman SGS                   |
| Wall treatment         | Boundary-layer solver                | Wall-resolved ( $y^+ < 1$ ) | WMLES ( $y^+ \approx 30$ )   |
| Wall model             | –                                    | –                           | Reichardt                    |
| Computational domain   | –                                    | $30c$                       | $30c$                        |
| Polynomial order       | –                                    | –                           | 2                            |
| Spanwise length        | –                                    | 2D                          | $0.2c$                       |
| Spanwise planes        | –                                    | –                           | 24                           |
| Parallelization        | Serial                               | MPI                         | MPI + OpenACC                |
|                        | Cost: seconds to a few minutes       | Cost: minutes to hours      | Cost: hours to days          |

Can benefit from proper HPC facilities

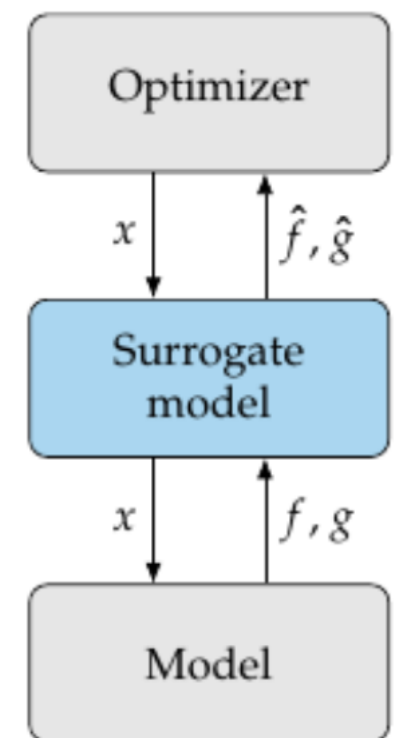
# Act II: Learning the model

# Act II: Learning the model

## Surrogate modeling, a solution

- Replace original model by an approximated and faster one
- Represents a fit to some underlying data
- Can remain “fixed”, or “updated” throughout optimization
- Naturally “smooths” the data when noisy
- Can be interesting to “simplify” the design space and better “understand” our model
- Can "blend" different efficiency/fidelity levels of data

NOTE: building the surrogate also needs computational time, and engineering time too! (requires validation). Building cost could explode if multiple design variables.





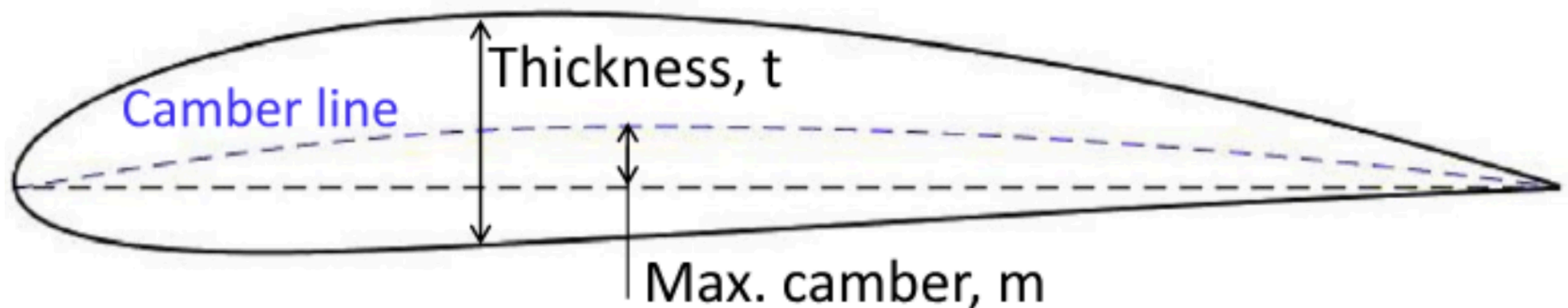
# Act II: Learning the model

## NACA 4 digit parametrization

NACA MPXX

- M is the maximum camber divided by 100
- P is the position of the maximum camber divided by 10
- XX is the thickness divided by 100

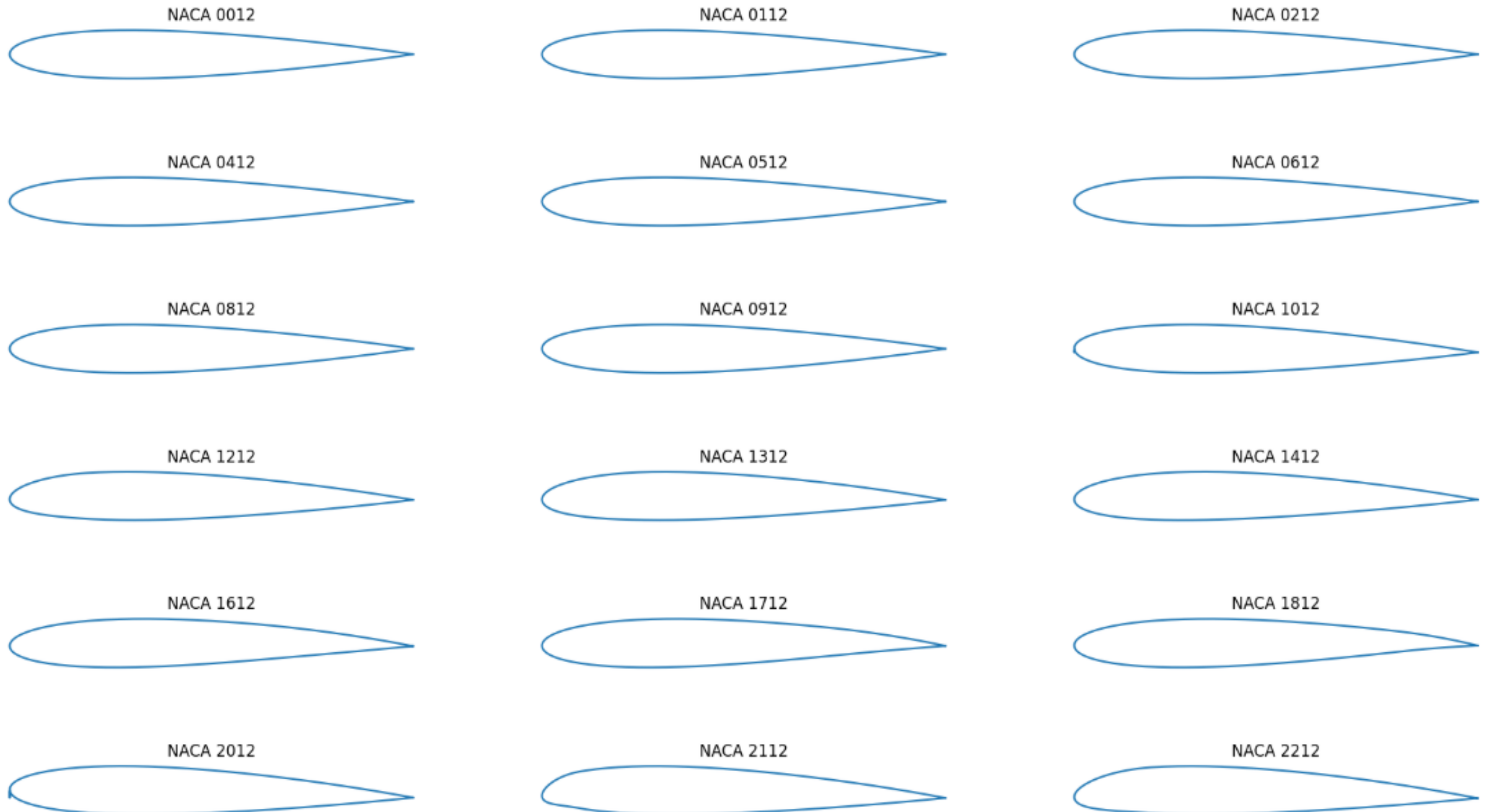
Do not panic! Formulations will be provided later





# Act II: Learning the model

## The NACA 4 digit family by example



# Act II: Learning the model

## Operating conditions (AoA, Re, M)

- The model is never a just map from geometry to  $C_L$ ,  $C_D$
- Operating conditions should always be taken into account, **and add extra dimensions!**

$$(m, p, t) \times (\alpha, Re, M) \rightarrow C_L, C_D$$

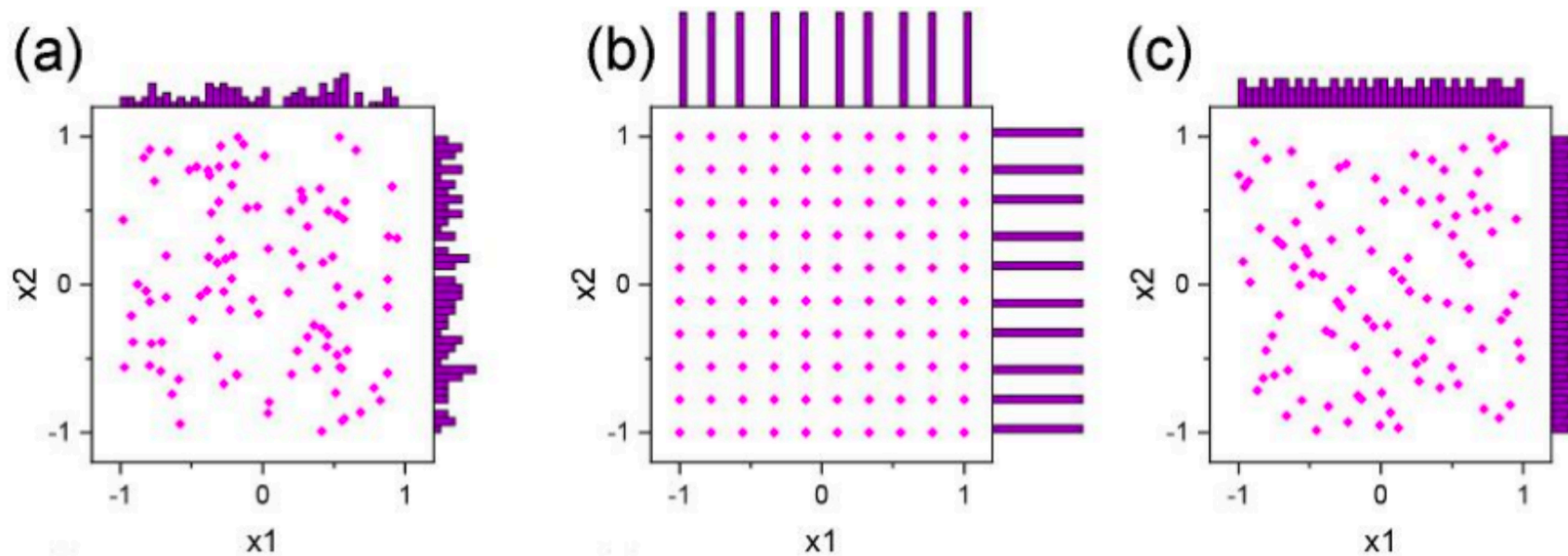
- Additionally, new physics appear:
  - Add Re,  $3 \sim 5 \times$  complexity, transition and drag scaling
  - Add Ma,  $10 \times$  complexity, compressibility and shocks

# Act II: Learning the model

## Sampling

Select evaluation points to construct the surrogate. Methods:

- (a) Random
- (b) Full factorial (FF)
- (c) Latin hypercube sampling (LHS)



# Act II: Learning the model

## POD/PCA on airfoil coordinates

We can use linear algebra, SVD decomposition:

$$X = \begin{bmatrix} \vdots & \vdots & \vdots \\ x_1 & x_2 & x_n \\ \vdots & \vdots & \vdots \end{bmatrix} = U \Sigma V^T = \begin{bmatrix} \vdots & \vdots & \vdots \\ U_1 & U_2 & U_m \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} \sigma_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_n \end{bmatrix} \begin{bmatrix} \vdots & \vdots & \vdots \\ V_1 & V_2 & V_n \\ \vdots & \vdots & \vdots \end{bmatrix}^T$$

- $X$  is the design space (if we divide by its variance we get PCA)
- $U$  are the POD/PCA modes (e.g., mode 1 represents thickness, mode 2 represents camber)
- $\Sigma V^T$  is the POD/PCA basis

This decomposition is **orthogonal and optimal**, that's what is letting us separate the modes!

# Act II: Learning the model

## The cost

Nothing forbids a linear combination of valid airfoils from being invalid:

$$\mathbf{a} = \bar{\mathbf{a}} + \sum_{i=1}^k c_i \mathbf{u}_i \quad \longrightarrow \quad \text{self-intersecting, negative thickness, wavy surfaces}$$

Shapes far from  $\bar{\mathbf{a}}$  need many modes. The manifold is curved; the subspace is flat.

Advantage. Project any new shape onto the truncated basis and measure what is left over:

$$r(\mathbf{a}) = \frac{\left\| (\mathbf{a} - \bar{\mathbf{a}}) - \mathbf{U}_k \mathbf{U}_k^T (\mathbf{a} - \bar{\mathbf{a}}) \right\|}{\left\| \mathbf{a} - \bar{\mathbf{a}} \right\|}$$

- $r$  small: the shape lies in the span of what the data has seen
- $r$  large: the shape is outside the training manifold

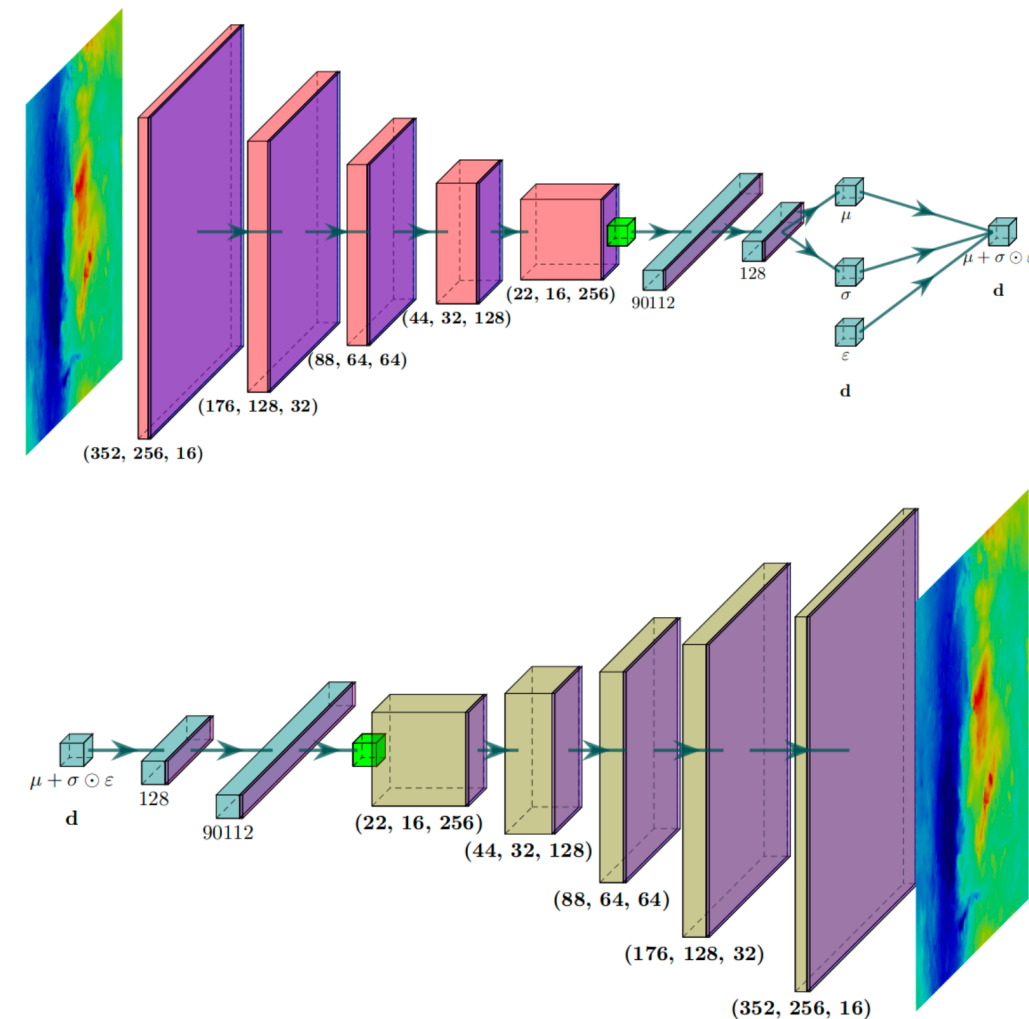
# Act II: Learning the model

## (Variational) autoencoders

- **Encoder:** function that transforms the input data to the latent space
- **Decoder:** function that recovers the input data from the latent space
- **Latent space:** embedding of a set of items within a manifold in which items resembling each other are positioned closer to one another

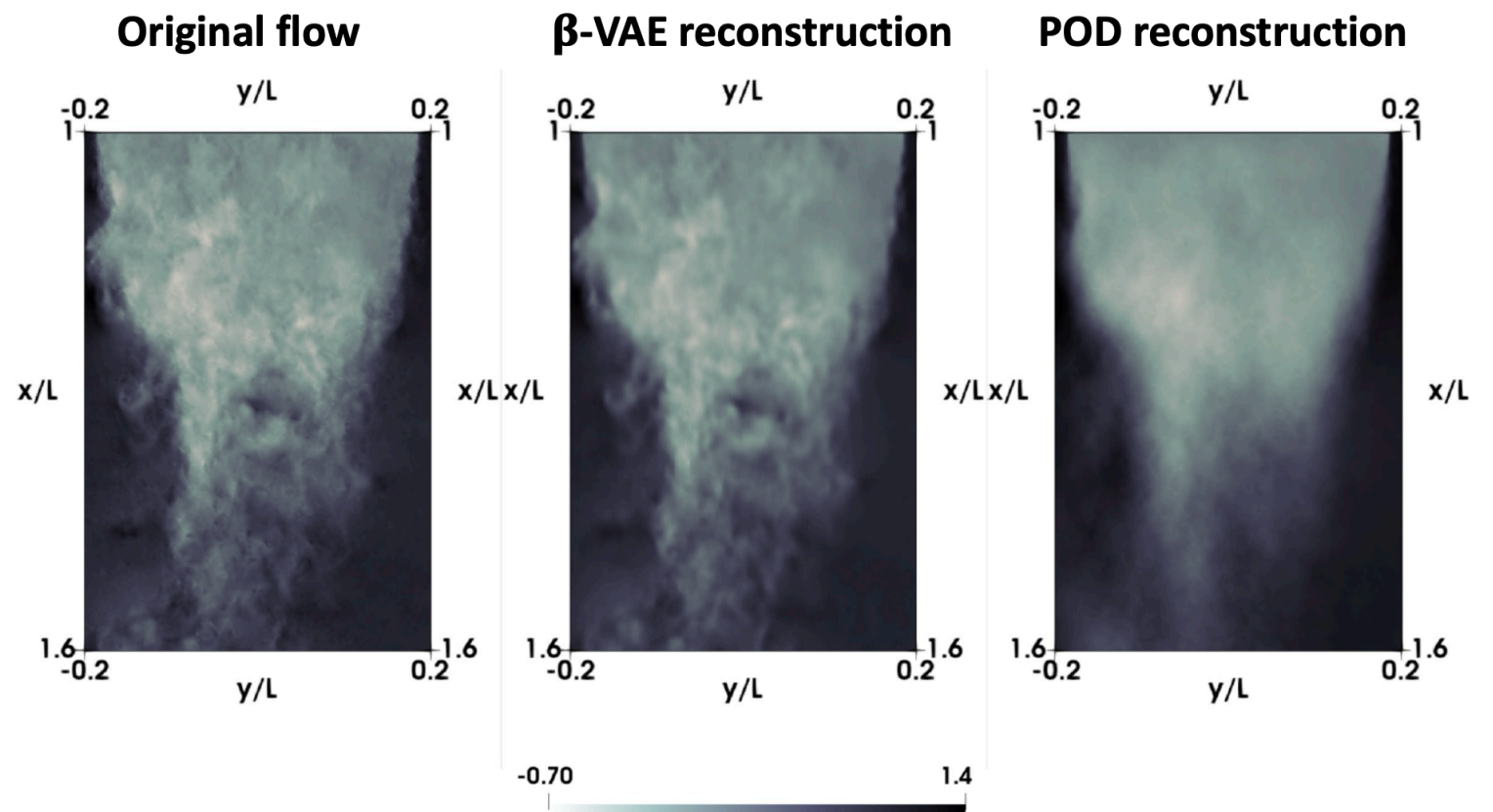
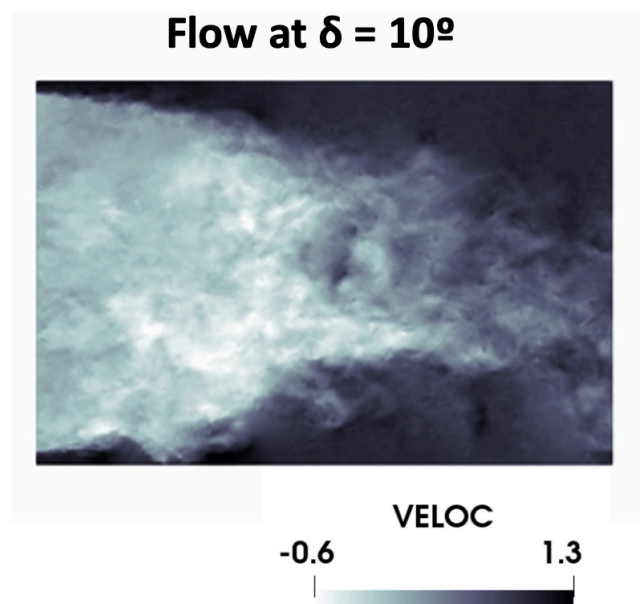
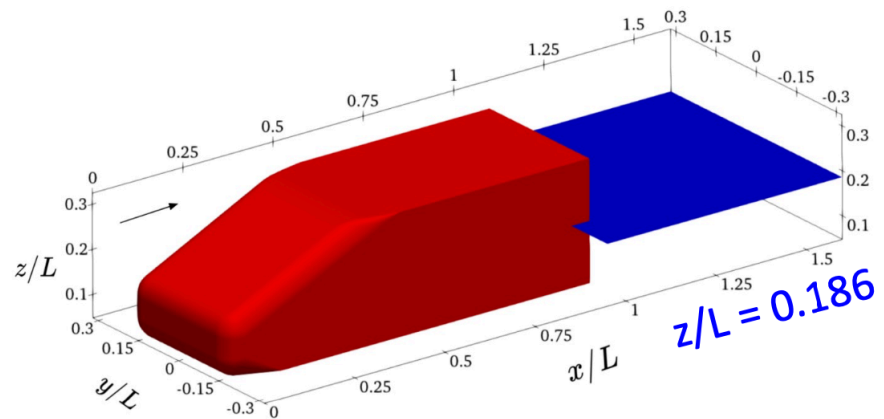
If the latent space is a *probabilistic function*, we talk about variational auto-encoders

- VAEs are highly non-linear and their modes are *not guaranteed to be orthogonal*
- In general, VAEs can recover *a much higher percentage of energy with much fewer modes* than POD.



# Act II: Learning the model

## VAEs, where it lands



- $\beta$ -VAE, 20 latent dimensions, 96.75% energy of the total flow
- POD, 20 modes, 40% of the total flow

*Caveat: 3D flow field; airfoil contours are nearly linear so the gap shrinks*



# Act II: Learning the model

## The 2x2

|                          | Algebraic                                      | Machine learning                                      |
|--------------------------|------------------------------------------------|-------------------------------------------------------|
| Compress<br>(input side) | POD/PCA                                        | Autoencoder, $\beta$ VAE                              |
| Fit<br>(output side)     | Linear regression,<br>RBF, Kriging             | MLP, KAN,<br>transformer                              |
|                          | <b>One solve</b> ,<br>deterministic,<br>unique | <b>Many steps</b> , not<br>unique, seed-<br>dependent |

- A linear autoencoder with bottleneck  $k$ , trained to minimise MSE, **converges to the span of the first  $k$  POD modes** (Bourlard & Kamp 1988; Baldi & Hornik 1989)
- Linear regression is an MLP with no hidden layer and identity activation. Stack ten linear layers and you still have linear regression.

***So the question is: is your manifold curved enough to pay for what you lose?***



# Act II: Learning the model

## Target choice

- Thin airfoil, small AoA:  $C_d \approx 0.006$
- Thick airfoil, near stall:  $C_d \approx 0.2$  (about x30 times larger)

Train on  $C_d$  with MSE:

$$\mathcal{L} = \frac{1}{N} \sum \left( C_d - \hat{C}_d \right)^2$$

An error of 0.01 at  $C_d \approx 0.2$  contributes the same loss as an error of at  $C_d \approx 0.006$ , where it is a 170% error. MSE optimises the cases you care least about.

***Train on  $\log_{10} C_d$  not on  $C_d$***

# Act II: Learning the model

## pyLOM in a nutshell

- Our python package for high-performance-computing and massively parallel reduced order models
- Parallel POD, DMD and SPOD based on a parallel (randomized) QR factorization algorithm
- Scalable using either CPU or GPU (AMD support experimental)
- CPU version can be installed with: **pip install pyLowOrder**
- GPU version can be installed with: **pip install pyLowOrder[cuda12x]**
- ML/AI algorithms implemented in Neural Network (NN) module: MLP,  $\beta$ -VAE, KAN and PINN
- NN module can be installed with: **pip install pyLowOrder[NN]**
- Compile code from source (github) to enhance CPU performance

**Act III: How much can I  
trust the model?**

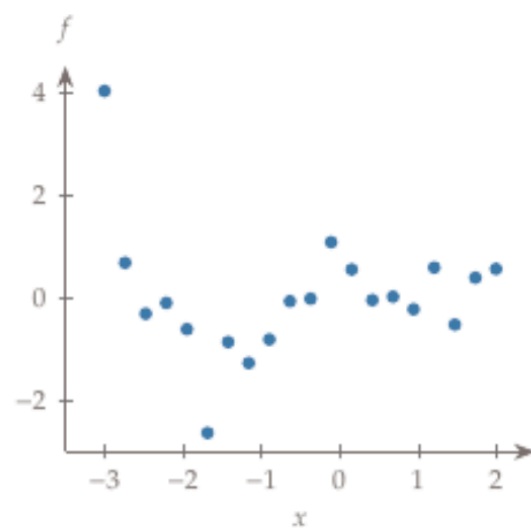
# Act III: How much can I trust the model?

## The basics

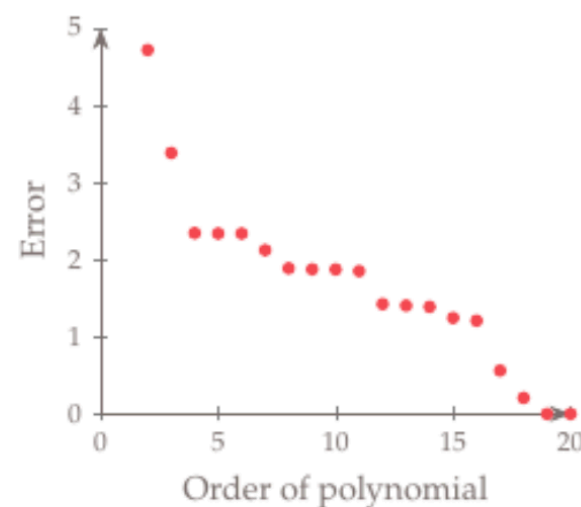
Once we build our surrogate, we should check if it does “work”

- Is training data properly selected?
- Is our choice of basis functions suitable for our surrogate?

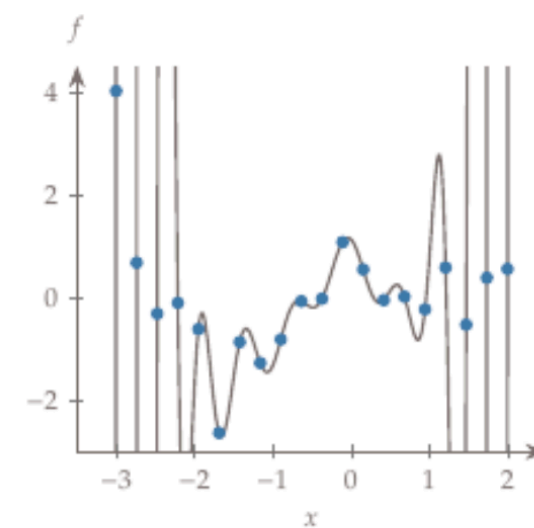
One danger: over-fitting



Training data



The error in fitting the data decreases with the order of the polynomial



A 19th-order polynomial fit to the data has low error but poor predictive ability

# Act III: How much can I trust the model?

## A simple strategy

A simple approach to check things are OK (solid for over-fitting):

- Randomly split your data into a training set and a validation set (e.g. 70% and 30%)
- Develop your model with training data set
- Assess an error metric on the validation data set
- Try different strategies, and pick the one minimizing the error
- Optionally: re-train your choice with all the data (i.e. without the split)

For general regression problems, one can also compute an error metric on the whole set and that's it (i.e. skipping cross-validation)

# Act III: How much can I trust the model?

## Where does the test set come from?

Example campaign: 200 airfoils  $\times$  10 angles of attack = 2000 rows.  
Now split randomly, 80/20

- NACA 2412 at AoA  $4.0^\circ$  (train) / NACA 2412 at AoA  $4.5^\circ$  (test)

Two rows. Nearly identical inputs. Nearly identical outputs. **Is this a good split?**

| Split strategy                 | $R^2$ on $C_l$ | Honest? |
|--------------------------------|----------------|---------|
| Random row split               | 0.998          | No      |
| Group split by airfoil         | 0.981          | Yes     |
| Group split + unseen thickness | 0.72           | Painful |

# Act III: How much can I trust the model?

## Metrics that lie

| Metric                      | On $C_d$ | Reads as           |
|-----------------------------|----------|--------------------|
| R <sup>2</sup>              | 0.994    | Excellent          |
| RMSE                        | 0.0021   | Tiny               |
| MRE                         | 31%      | Unusable at cruise |
| Relative error on $C_l/C_d$ | 38%      | Catastrophic       |

- Same model, four ways of scoring it
- R<sup>2</sup> measures variance explained. **Drag variance is dominated by the stalled cases.** You're being lied in your face
- Errors add
- Report the metric your decision depends on, not the one your library returns by default



# Act III: How much can I trust the model?

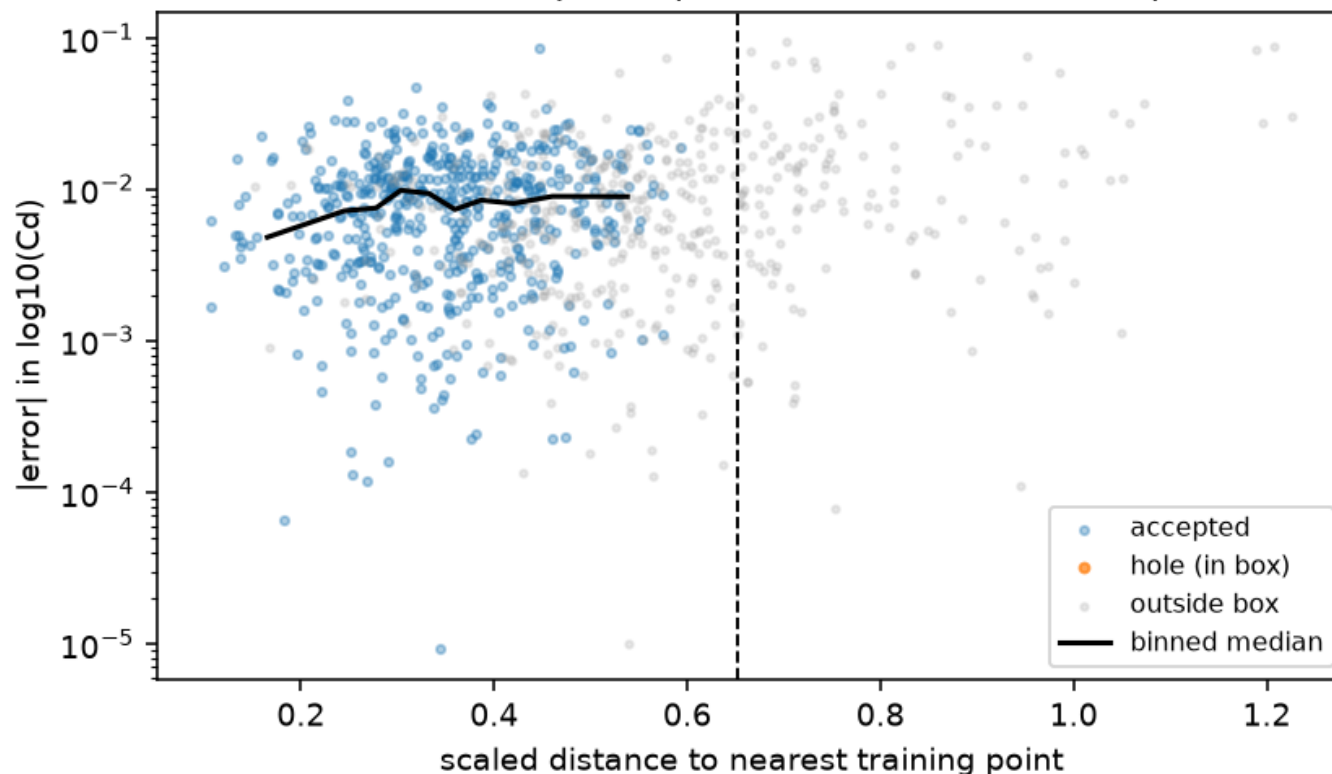
## Interpolation vs extrapolation

$$d(\mathbf{x}) = \min_i; \|\mathbf{x} - \mathbf{x}_i\| \text{scaled} \quad \text{accept if } d(\mathbf{x}) < \tau$$

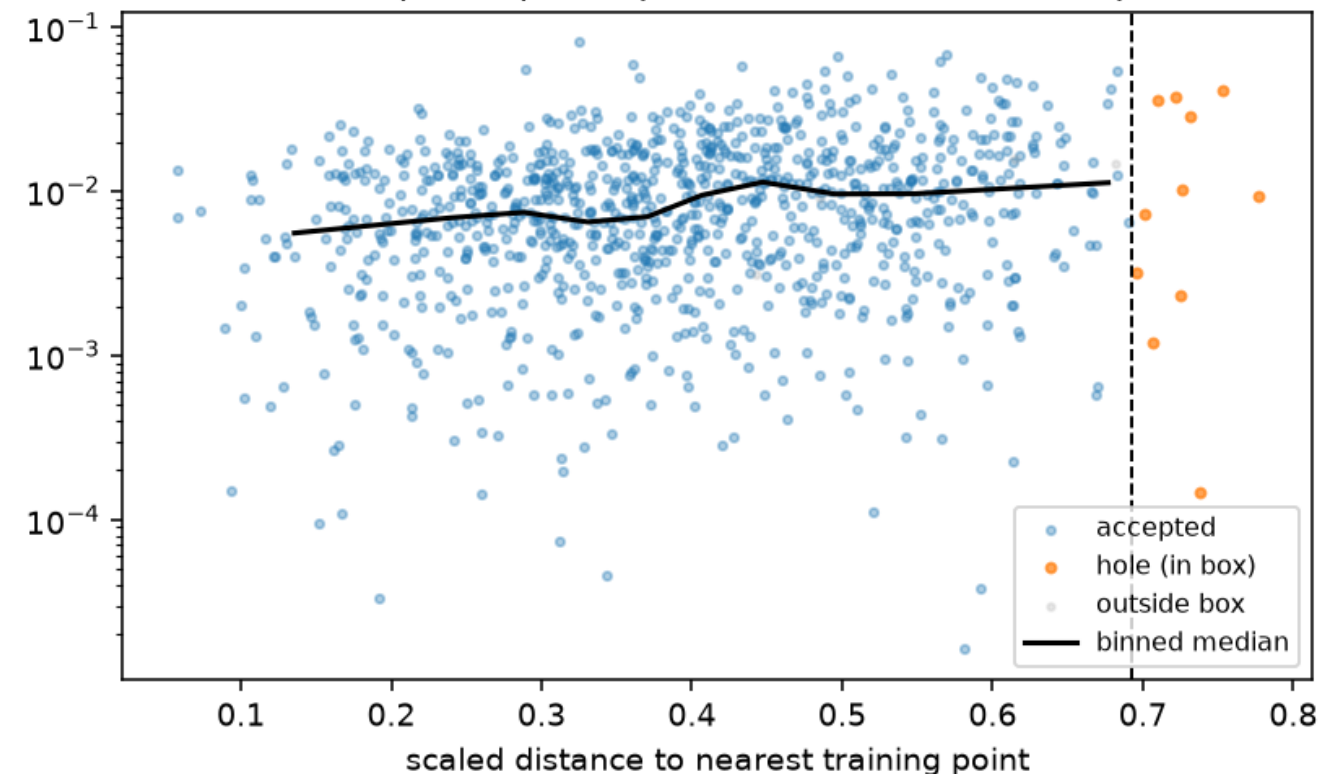
Nearest neighbour: easily implemented through a KD-tree

A flat black line means the envelope is not informative here

reference -- probe points, all inside the clamp



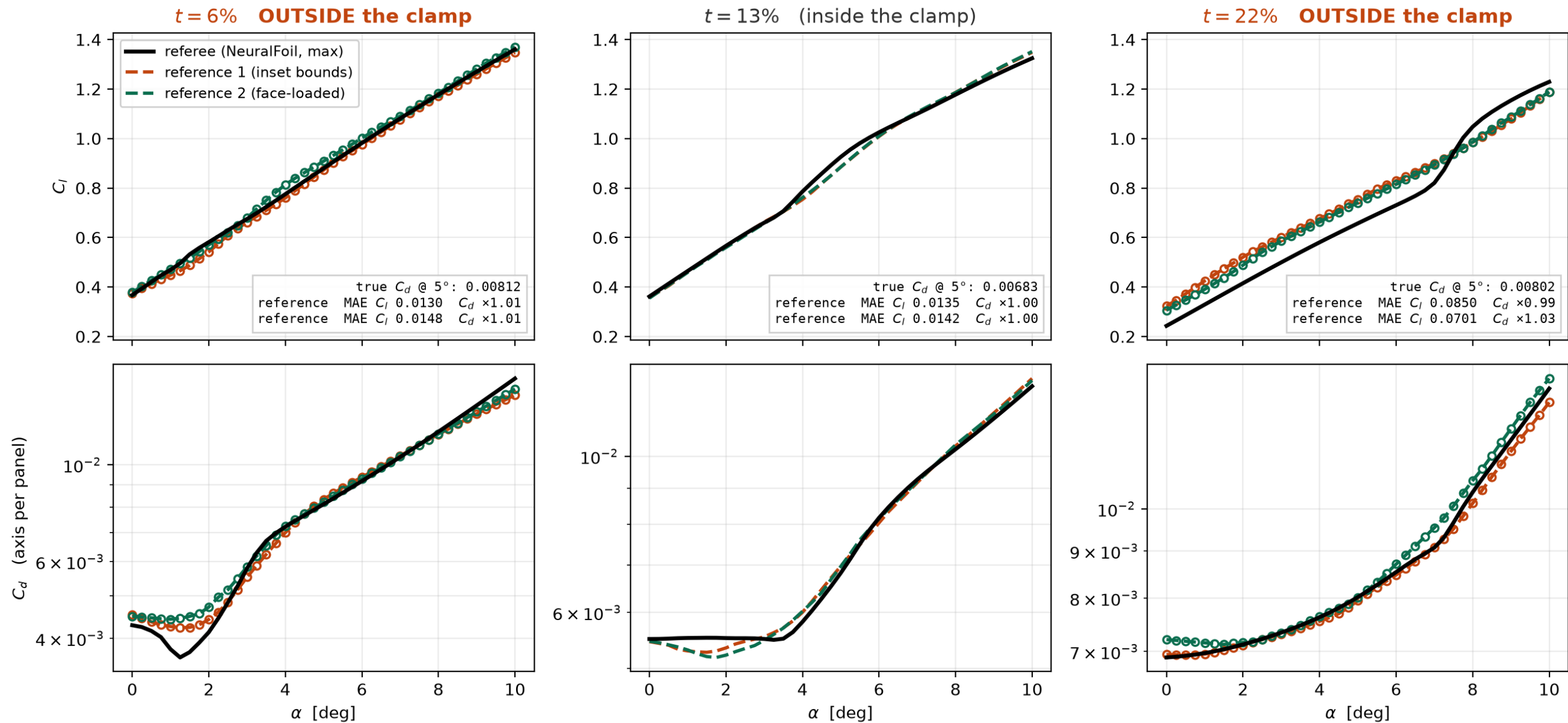
exemplar -- probe points, all inside the clamp





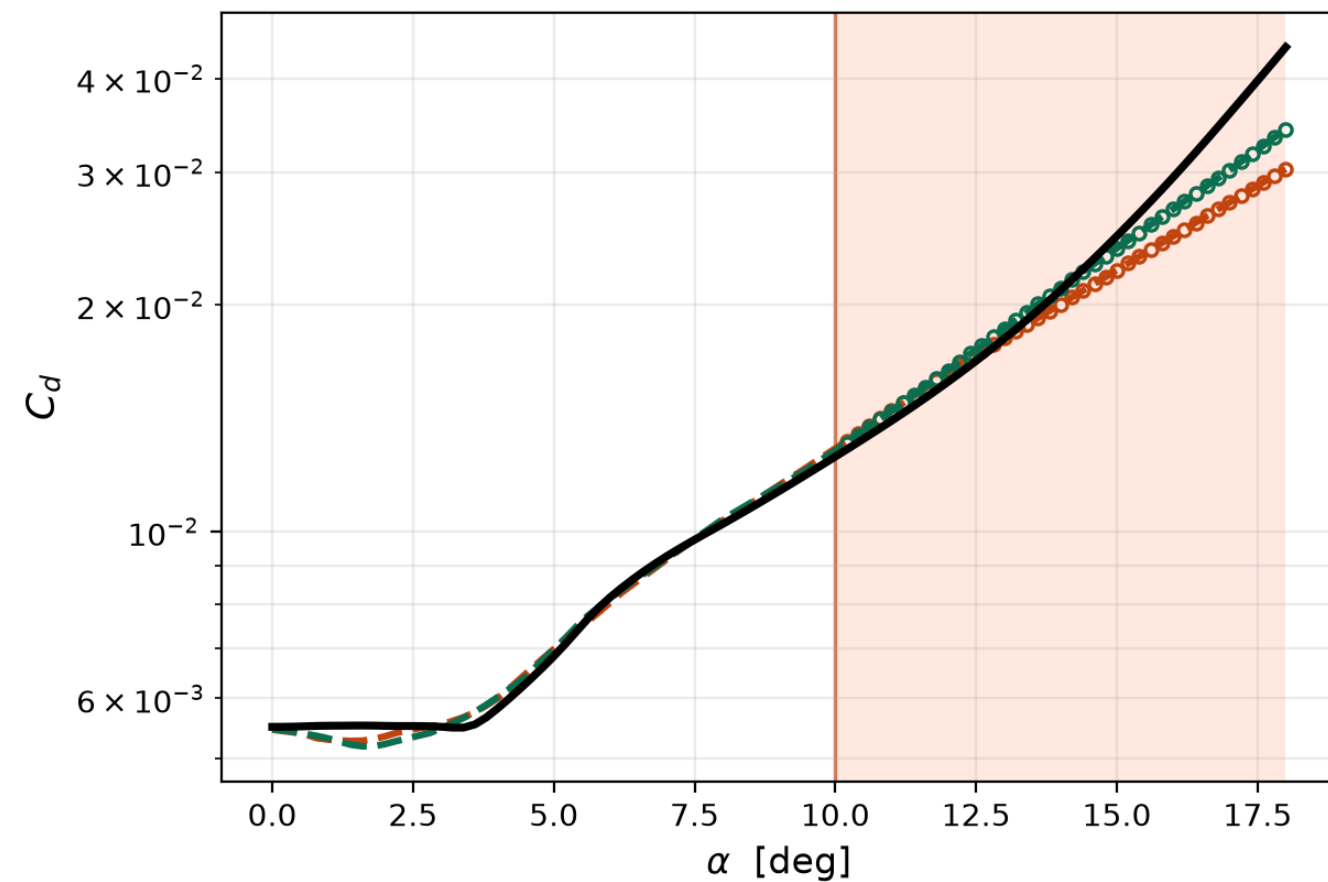
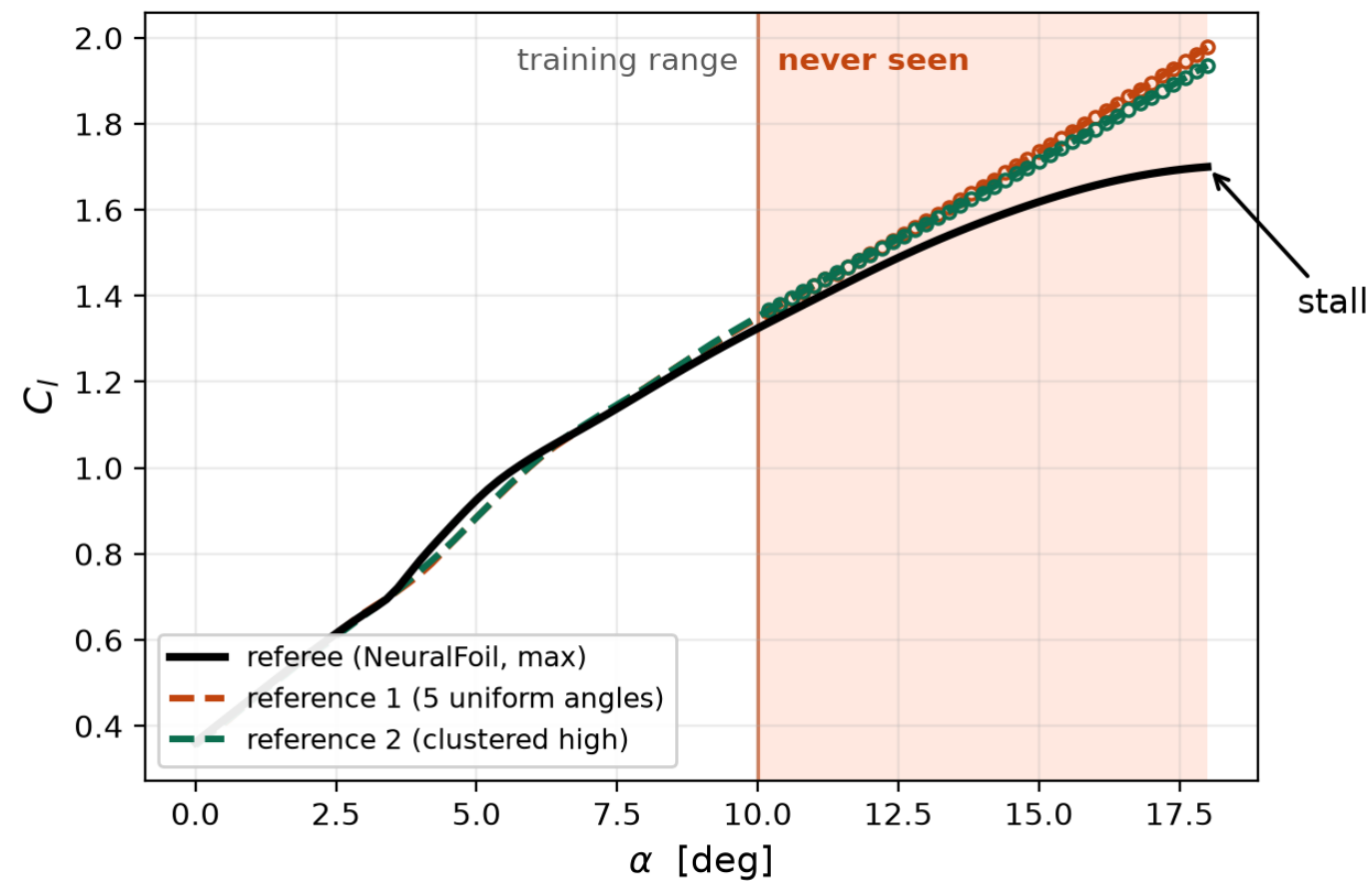
# Act III: How much can I trust the model?

## Failure 1: unseen geometry



# Act III: How much can I trust the model?

## Failure 2: unseen physics



# Act III: How much can I trust the model?

## Confidently wrong

A point estimate carries no information about its own validity. This is a property of the method, not a bug in the training. So the envelope must ship with the model.

A surrogate is not a function. It is a function plus a domain.

Some ways to draw that domain:

- **Bounding box:** trivial, basic, always available
- **Convex hull / POD projection residual:** linear algebra, easy to compute
- **Ensemble disagreement / Kriging  $\sigma$ :** needs training, graded

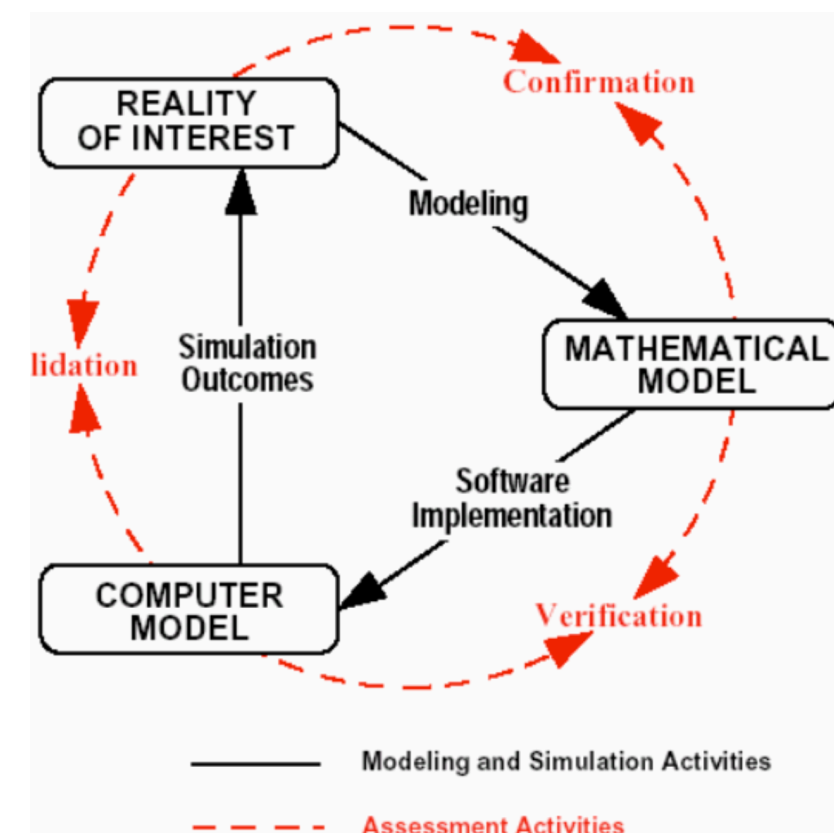
# Act III: How much can I trust the model?

## Does it work?

*Can a lightweight Machine Learning model trained on simple geometric parameters accurately predict the lift and drag coefficients of an unseen airfoil in milliseconds?*

Yes if:

- **Verification:** *getting the physics right* when implementing a model
- **Validation:** *getting the right physics* when applying a model



# **Act IV: Using the model**

# Act IV: Using the model

## The idea behind

Intuitively,

- Given a set of points  $x_i$ , evaluate the objective function  $f$  to obtain  $y_i$
- Obtain a measure of how “good” the evaluation  $y_i$  is from the minimum
- Generate a new set of points  $x_i$  to evaluate  $f$  and rinse and repeat

# Act IV: Using the model

## Recall, evaluations per search strategy

### Gradient-based methods

- $\nabla f$  must exist, is known and can be evaluated (reasonably)
- Fast approach towards the minimum with minimal evaluations: 10 to  $10^2$
- Methods are generally *mathematical*

### Gradient-free methods

- $\nabla f$  is unknown and only  $f$  can be evaluated
- We can afford a reasonable number of evaluations of  $f$ : up to  $10^4$
- Methods are generally *heuristic*



# Act IV: Using the model

## Reframing the problem

Every method  $s$  returns the same kind of object:

$$\mathbf{x}^* \in \mathbb{R}^n$$

**One point. One problem. One answer.**

What if the output were instead a function?

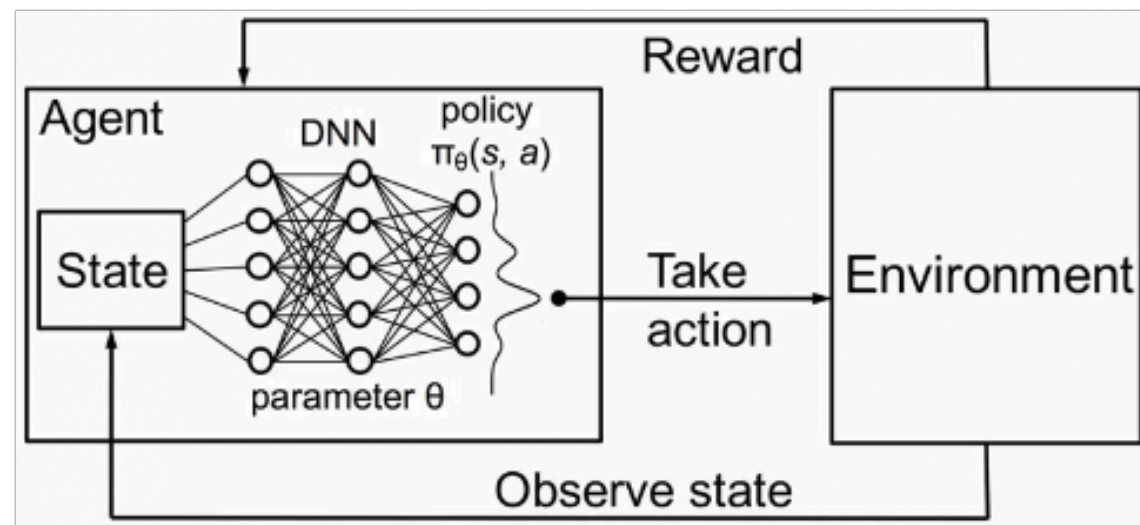
$$\pi : \text{situation} \longmapsto \text{design move}$$

**A recipe versus a cook.** The recipe is a better answer if you are making that one dish once. The cook is a better answer if you are running a kitchen.



# Act IV: Using the model

## Agent-environment interaction



# Act IV: Using the model

## Formalisms

A decision problem is a Markov decision process if it is fully described by:

- $\mathcal{S}$ , states, the section:  $(m, p, t)$
- $\mathcal{A}$ , actions, a change to the states:  $(\Delta m, \Delta p, \Delta t)$
- $P$ , transition, deterministic: you asked for it, you get it
- $R$ , reward, one scalar
- $\gamma$ , discount, how much a future reward is worth now
- Markov = the current state is a sufficient statistic. How you got here does not matter.

What the agent is trying to maximise is not the reward, the return:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Three objects, and all of RL is combinations of them:

- $\pi(a \mid s)$  policy: what to do. *The thing you actually want*
- $V^\pi(s)$  value: expected return from  $s$  if you follow  $\pi$
- $Q^\pi(s, a)$  action-value: expected return from  $s$  after doing  $a$ , then following  $\pi$

# Act IV: Using the model

## Policies

Your agent tries NACA 2412. It scores well.

*What should it do next?*

- **Try it again?** You know what you get. And you will never find anything better.
- **Try something else?** It will usually be worse. Occasionally it will not.

Every RL algorithm is, underneath, a policy for making this choice. How the two algorithms answer it:

- **PPO:** keep the policy's output *stochastic*; add an entropy bonus to the objective so that being too certain is penalised.
- **SAC:** put entropy in the objective itself, the agent is rewarded for acting as randomly as it can while still doing well.

# **Act IV: Using the model**

## **Rewards**

Your agent may change camber, camber position, and thickness.

It gets rewarded for producing a good wing section.

**Write that reward. One line.**

# Act IV: Using the model

## Rewards

**Sparse:** reward only at the end of the episode.

- Says exactly what you want, and nothing about how to get there
- Correct by construction
- Sixty-four moves of silence, then one number. Very hard to learn from

**Dense:** reward at every step, e.g. the improvement in L/D since the last move.

- Learns far faster
- And injects your opinion about how the goal should be reached

Audit any reward before you run it:

1. What is the argmax, if the model were perfect?
2. What term is unbounded?
3. What is not penalized?
4. Can the reward be gained without doing the intended thing?
5. Where is the model wrong, and could you tell from inside your own data?

**Act V: RL, what it finds,  
where it wins**

# Act V: RL, what it finds, where it wins

## Policy versus point solution

Cost of one design problem:

- Differential evolution: ~800 evaluations per new condition
- Trained policy: ~100,000 training evaluations, one forward pass

Solve for the crossover:

$$N^* = \frac{100,000}{800} = \mathbf{125}$$

Who wins?

- 1 problem to solve: DE, overwhelmingly
- 10 problems to solve: DE, comfortably
- 125 problems to solve: a tie
- 10,000 problems to solve: the policy, overwhelmingly
- a control loop at 1 kHz: the policy is the only option

So the question is never "*is RL better?*". It is: *how many times are you going to ask?*



# Act V: RL, what it finds, where it wins

## Where this pays

RL is the right/wrong tool when:

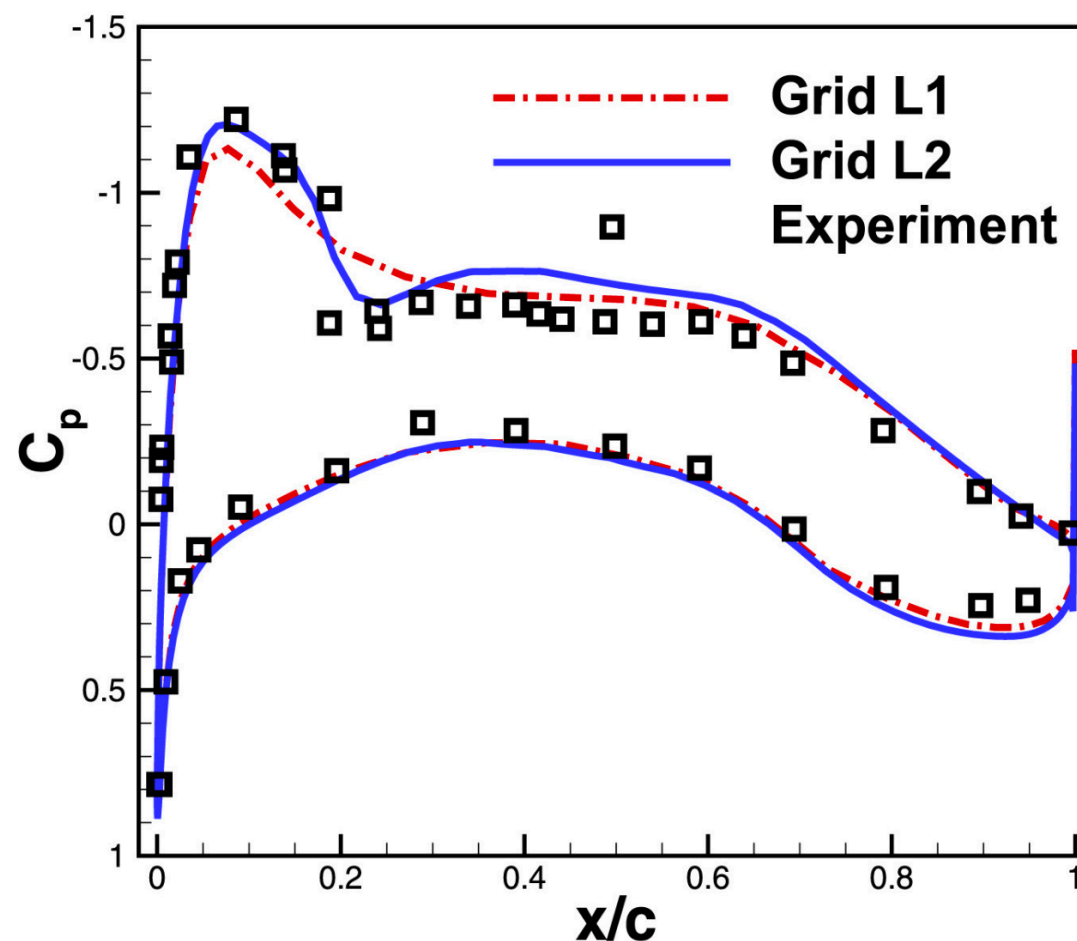
| Right tool                             | Wrong tool                               |
|----------------------------------------|------------------------------------------|
| You ask multiple times                 | You ask once                             |
| The problem is complex                 | The problem is smooth and differentiable |
| The dimensionality is high             | The dimensionality is low                |
| Evaluations are costly                 | Evaluations are cheap and exact          |
| You need a policy (algorithm agnostic) | You need a reproducible answer           |



# Act V: What the search finds

## One idea: multi-fidelity

Combine the benefits of several aerodynamic models in a multi-fidelity approach: lower fidelity/ies to explore the design space, and higher fidelity/ies to refine search/increase accuracy



# **Act VI: Final thoughts**

# Act VI: Final thoughts

## Wrapping up

Surrogate-based optimization is useful for expensive or noisy optimization problems.

Each step should be carefully executed and adapted to the tackled problem:

- **Sampling:** aiming at good coverage with a few points
- **Modeling:** when one can opt for e.g. use linear or non-linear regression models, and involved algebra operations and available system solvers
- **Cross-validation:** an important step

*What surrogates never do:* they don't discover physics they were never shown.

# Act VI: Final thoughts

## Wrapping up

*The optimizer is not looking for where your model is wrong. It is looking for where your model is highest.*

- Those are different places.
- Until they are not. And from inside the model, you have no way of telling which case you are in.

*An optimizer is a machine for finding the most flattering error in your data.*

- And note what this is not about: Nothing on this slide mentions reinforcement learning. Differential evolution does this too. Gradient descent does this too. Anything that searches hard enough will find it. *RL is not the villain. RL is just very good at searching.*

# Act VI: Final thoughts

## Research questions

- Can a lightweight Machine Learning model trained on simple geometric parameters accurately predict the lift and drag coefficients of an unseen airfoil in milliseconds?
- At what point does an AI surrogate model begin to fail when predicting flow behaviour outside of its training envelope?
- How does the optimization workflow of a Reinforcement Learning agent change when it is tasked with modifying a static physical blueprint (Aerodynamic Shape Optimization) versus responding dynamically to a live fluid environment (Active Flow Control)?
- What are the trade-offs in computational training time and predictive accuracy when using Reinforcement Learning as an optimization framework compared to traditional, physics-only engineering methods?

# Act VI: Final thoughts

## Thoughts for tonight

A fixed budget of 2000 aerodynamic evaluations, four inputs  $(m, p, t, \alpha)$ , and one question:

**how do you spend it?**

In teams you'll design an evaluation campaign, generate a dataset and train a model on it.

Food for thought:

- How many shapes, how many angles each?
- Where do you put the bounds?
- Sampling: grid, random, or something cleverer?
- What will your dataset be bad at?

**Q & A**